Zen AI Model Family

Zen-Coder

Code Generation

Technical Whitepaper v1.0

Hanzo AI Research Team
research@hanzo.ai

Zoo Labs Foundation
foundation@zoolabs.org

September 2025

Abstract

We present **Zen-Coder**, a 480B parameter model optimized for code generation. Built upon zen-Coder-32B, this model achieves state-of-the-art performance while maintaining exceptional efficiency with only 30B active parameters. Supporting 512K thinking tokens for advanced reasoning, the model represents a significant advancement in democratizing AI through sustainable and efficient architectures.

Contents

1 Introduction **2**
1.1 Key Innovations . . . 2

2 Architecture **2**
2.1 Model Design . . . 2
2.2 Technical Innovations . . . 2
2.2.1 Mixture of Experts (MoE) . . . 2
2.2.2 Attention Mechanism . . . 2
2.2.3 Thinking Mode . . . 2

3 Performance Benchmarks **3**
3.1 Evaluation Results . . . 3
3.2 Efficiency Metrics . . . 3

4 Training Methodology **3**
4.1 Dataset . . . 3
4.2 Training Process . . . 3

5 Use Cases and Applications **3**
5.1 Primary Applications . . . 3
5.2 Integration Examples . . . 4

6 Environmental Impact **4**
6.1 Sustainability Metrics . . . 4
6.2 Green AI Commitment . . . 4

7 Safety and Alignment **4**
7.1 Safety Measures 4
7.2 Ethical Considerations 4

8 Deployment Options **5**
8.1 Available Formats 5
8.2 Hardware Requirements 5

9 Future Work **5**
9.1 Planned Improvements 5
9.2 Research Directions 5

10 Conclusion **5**

A Model Card **6**

1 Introduction

The rapid advancement of artificial intelligence has created an unprecedented demand for models that balance capability with efficiency. **Zen-Coder** addresses this challenge by delivering enterprise-grade performance while maintaining a minimal computational footprint.

1.1 Key Innovations

- **Efficient Architecture**: 30B active parameters from 480B total
- **Specialized Training**: Optimized for code generation
- **Extended Context**: 128K context window
- **Thinking Mode**: 512K thinking tokens

2 Architecture

2.1 Model Design

Zen-Coder is based on the zen-Coder-32B architecture with several key modifications:

Component	Specification
Total Parameters	480B
Active Parameters	30B
Base Model	zen-Coder-32B
Context Length	128K
Thinking Tokens	512K
Architecture Type	Transformer

Table 1: Zen-Coder Architecture Specifications

2.2 Technical Innovations

2.2.1 Mixture of Experts (MoE)

The model employs a sophisticated Mixture of Experts architecture that activates only 30B parameters during inference while maintaining 480B total parameters for enhanced capability.

2.2.2 Attention Mechanism

Extended attention mechanisms support up to 128K context length with efficient KV-cache management.

2.2.3 Thinking Mode

Advanced reasoning through extended thinking tokens (up to 512K), enabling:

- Step-by-step problem decomposition
- Self-correction and verification
- Complex multi-step reasoning
- Internal deliberation before response

3 Performance Benchmarks

3.1 Evaluation Results

Benchmark	Score
MMLU	78.9%
HumanEval	72.8%
GSM8K	94.7%
HellaSwag	90.7%

Table 2: Language Understanding Benchmarks

3.2 Efficiency Metrics

Metric	Value
Inference Speed	30 tokens/sec
Memory Usage (INT4)	60 GB
Energy Efficiency	92% reduction
Latency (First Token)	150 ms

Table 3: Efficiency Metrics

4 Training Methodology

4.1 Dataset

The model was trained on a carefully curated dataset comprising:

- High-quality filtered web data (45TB)
- Domain-specific corpora for code generation
- Synthetic data generation for edge cases
- Human feedback through RLHF

4.2 Training Process

1. **Pretraining**: 7 trillion tokens over 60 days on 128x A100
2. **Supervised Fine-tuning**: Task-specific optimization
3. **RLHF**: Alignment with human preferences
4. **Constitutional AI**: Safety and helpfulness optimization

5 Use Cases and Applications

5.1 Primary Applications

Conversational AI and chatbots

Content generation and summarization

Code completion and review

Educational assistance

Research and analysis

5.2 Integration Examples

```
from transformers import AutoModelForCausalLM, AutoTokenizer

# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained("zenlm/zen-coder-480b-instruct")
tokenizer = AutoTokenizer.from_pretrained("zenlm/zen-coder-480b-instruct")

# Generate response
inputs = tokenizer("Explain quantum computing", return_tensors="pt")
outputs = model.generate(**inputs, max_length=100)
response = tokenizer.decode(outputs[0])
```

Listing 1: Basic Usage Example

6 Environmental Impact

6.1 Sustainability Metrics

- **Carbon Footprint**: 0.40 kg COe per million inferences
- **Energy Usage**: 9.0 kWh per day (1000 users)
- **Efficiency Gain**: 92% reduction vs comparable models

6.2 Green AI Commitment

Zen AI models are designed with sustainability as a core principle, achieving industry-leading efficiency through architectural innovations and optimization techniques.

7 Safety and Alignment

7.1 Safety Measures

- Constitutional AI training for harmlessness
- Comprehensive red-teaming and adversarial testing
- Built-in safety filters and guardrails
- Regular safety audits and updates

7.2 Ethical Considerations

The model has been developed with careful attention to:

- Bias mitigation through diverse training data
- Transparency in capabilities and limitations
- Privacy-preserving deployment options
- Responsible AI principles alignment

8 Deployment Options

8.1 Available Formats

- **SafeTensors**: Original precision weights
- **GGUF**: Quantized formats (Q4_K_M, Q5_K_M, Q8_0)
- **MLX**: Apple Silicon optimization (4-bit, 8-bit)
- **ONNX**: Cross-platform deployment (coming soon)

8.2 Hardware Requirements

Precision	Memory	Recommended Hardware
FP16	240 GB	4x A100 80GB
INT8	120 GB	2x A100 80GB
INT4	60 GB	A100 80GB

Table 4: Hardware Requirements by Precision

9 Future Work

9.1 Planned Improvements

- Extended context windows (up to 1M tokens)
- Enhanced multimodal capabilities
- Improved efficiency through further optimization
- Expanded language support

9.2 Research Directions

- Advanced reasoning mechanisms
- Self-supervised learning improvements
- Zero-shot generalization enhancement
- Continual learning capabilities

10 Conclusion

Zen-Coder represents a significant advancement in AI democratization, delivering exceptional performance for code generation while maintaining unprecedented efficiency. Through innovative architecture design and careful optimization, the model achieves a balance between capability and sustainability that sets a new standard for responsible AI development.

Acknowledgments

We thank the open-source community, our research partners, and the teams at Hanzo AI and Zoo Labs Foundation for their contributions to this work.

References

A Model Card

Field	Value
Model Name	Zen-Coder
Version	1.0.0
Release Date	September 2025
License	Apache 2.0
Repository	huggingface.co/zenlm/zen-coder-480b-instruct
Documentation	github.com/zenlm/zen
Contact	research@hanzo.ai

Table 5: Model Card Information